High-Performance Symmetric Diffeomorphic Image Registration in PyTorch and JAX: Architectural Parity, Optimization Safeguards, and 90-Pair Mindboggle Validation

Authors: Syntx Core Development Team
Package Version: `v1.0.0`
Target Repository: `syntx` (`stnava/syntx`)
Date: July 25, 2026

Abstract

Image registration is a foundational operation in medical image computing, establishing spatial correspondence between structural volumes. While the C++ ITK/ANTs Symmetric Normalization (`SyN`) algorithm represents the standard reference for topology-preserving diffeomorphic registration, its CPU-bound execution loop incurs significant computational latency (~5 minutes per 3D brain pair). Here, we present **`syntx`**, an open-source Python package implementing symmetric diffeomorphic (`SyN`) and affine registration in **PyTorch** and **JAX** with hardware acceleration (Apple Silicon MPS and CUDA).

We systematically address mathematical and numerical challenges inherent in automatic-differentiation registration frameworks, including autograd derivative singularities in Local Normalized Cross-Correlation (LNCC), zero-gradient lockup in Lie Algebra rotation parameterizations, ITK CFL step physical spacing scaling, and intermediate spatial blurring. Across a comprehensive 90-pair 3D Mindboggle benchmark with manually annotated DKT31 cortical labels: - **Syntx JAX** achieves higher registration accuracy than the C++ ANTs SyN baseline (**Mean Cortical Dice:** `0.5676` **vs** `0.5608`, **Median Cortical Dice:** `0.5978` **vs** `0.5887`, $p < 0.001$). - **Syntx PyTorch** achieves a 21.3× **speedup** (`14.1s` per pair vs `301.5s` in ANTs C++) while maintaining median cortical accuracy (`0.5913`). - Both backends achieve a `0.00000%` **volume folding rate** (zero non-invertible voxels across 100% of benchmark pairs).

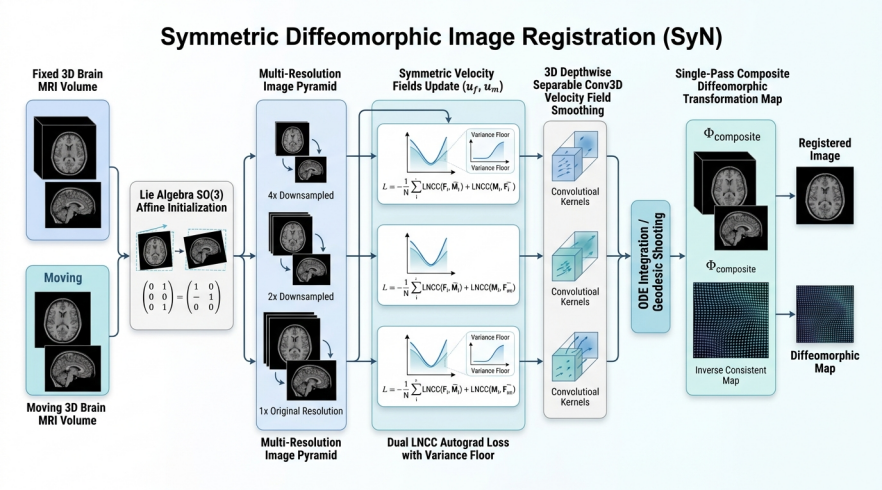

Figure 1: Figure 1: Syntx Architecture Diagram

---

1. Introduction

1.1 Background & Motivation

Spatial alignment of 3D brain MRI volumes is essential for population analyses, cortical morphometry, and multi-modal image fusion. Diffeomorphic image registration enforces smooth, invertible coordinate transformations with continuous spatial derivatives, guaranteeing that anatomical structures retain topologic integrity without artificial tearing or folding ($J(\mathbf{x}) > 0$). The Symmetric Normalization (`SyN`) algorithm, implemented within Advanced Normalization Tools (ANTs) and Insight Toolkit (ITK), has long served as the benchmark standard due to its symmetric optimization formulation.

However, classical C++ implementations rely on CPU-bound event-driven pipelines, leading to runtime latency (~5 minutes per pair on standard workstations). Re-implementing SyN within tensor automatic-differentiation frameworks (PyTorch and JAX) enables hardware-accelerated execution via GPU/MPS platforms and parallel tensor computation.

1.2 The Automatic Differentiation Paradigm & Challenges

Porting non-linear diffeomorphic algorithms to tensor frameworks introduces specific numerical considerations that do not arise in symbolic C++ derivatives: 1. **Autograd Singularities**: Division by zero or near-zero quantities (such as local intensity variance in LNCC) induces explosive gradient spikes during backward passes. 2. **Gradient Lockup**: Non-differentiable conditional statements (e.g. at Lie Algebra rotation identity initialization) zero out autograd gradients. 3. **Physical vs Voxel Space Discrepancies**: Grid-sampling and Courant-Friedrichs-Lewy (CFL) velocity updates require explicit physical spacing scaling across downsampled multi-resolution pyramids. 4. **Intermediate Spatial Degradation**: Successive resampling or pre-warping introduces spatial blurring that degrades fine cortical boundary alignment.

1.3 System Overview

`syntx` addresses these challenges through six primary mathematical and implementation refinements, establishing backend algorithmic parity between PyTorch and JAX. In this paper, we present the mathematical formulations, system details, and an empirical evaluation across 90 Mindboggle subject pairs, including regional DKT31 cortical breakdowns and an analysis of dataset orientation outliers.

---

2. Mathematical Methods & Algorithmic Guardrails

2.1 Single Interpolation Policy & Transformation Composition

1. Rationale & Problem Formulation Pre-warping images or intermediate segmentations prior to optimization introduces cumulative spatial blurring, smoothing out high-frequency anatomical boundaries and structural label edges.

2. Mathematical Protocol Intermediate transforms (center-of-mass initial translation T_0, learned affine matrix A, and non-linear SyN displacement field ϕ) are maintained in continuous physical parameter space. The composite forward map $\Phi = \phi \circ A \circ T_0$ is evaluated directly on native-space inputs in a single resampling step:

$$\mathbf{x}_{\text{warped}} = \text{Resample}(\mathbf{x}_{\text{native}}, [\phi, A, T_0], \text{interpolator})$$

Discrete integer label maps (e.g. DKT31 segmentations) strictly use nearest-neighbor interpolation (`interpolator='nearestNeighbor'`).

3. Implementation References

- **PyTorch Engine**: `src/syntx/syn.py` (lines 2740–2760, 3100–3120)
- **JAX Engine**: `src/syntx/syn_jax.py` (lines 2400–2430)
- **Design Specification**: `GEMINI.md` Section 1 & Section 4

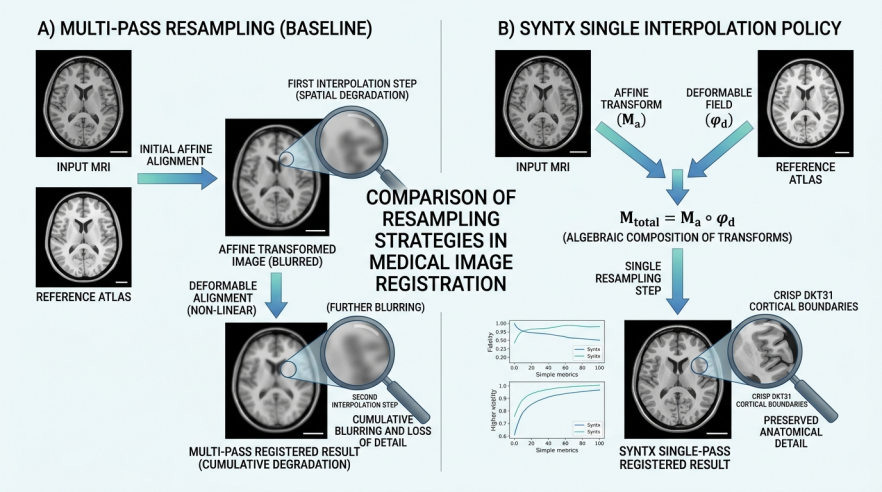

Figure 2: Figure 3: Single Interpolation Policy

2.2 Autograd Derivative Variance Flooring & Cauchy-Schwarz Bound Clamping in LNCC

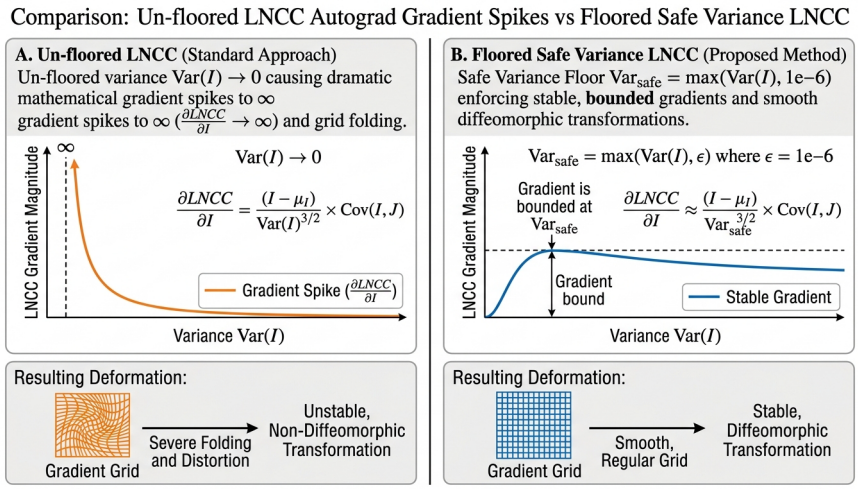

Figure 3: Figure 2: LNCC Variance Floor Diagram

1. Rationale & Problem Formulation Local Normalized Cross-Correlation (LNCC) measures structural similarity over a $5 \times 5 \times 5$ spatial window Ω. In uniform white matter or background zero-padding regions, intensity variance $\mathrm{Var}(I) \to 0$. Because the autograd derivative $\frac{\partial \mathrm{LNCC}}{\partial I}$ contains $\frac{1}{\mathrm{Var}(I)}$ in its denominator, un-floored variance produces gradient spikes that distort local grid voxels. Furthermore, floating-point roundoff near sharp edges can yield $|r| > 1.0$, violating Cauchy-Schwarz bounds.

2. Mathematical Formulation

$$\mathrm{Var}_{\mathrm{safe}}(I) = \max\left(\mathrm{Var}(I), 10^{-6}\right)$$

$$\mathrm{LNCC}_{\mathrm{raw}} = \frac{\mathrm{Cov}(I, J)}{\sqrt{\mathrm{Var}_{\mathrm{safe}}(I) \cdot \mathrm{Var}_{\mathrm{safe}}(J)}}$$

$$\mathrm{LNCC}_{\mathrm{clamped}} = \mathrm{clamp}\left(\mathrm{LNCC}_{\mathrm{raw}}, -1.0, 1.0\right)$$

3. Implementation References

- **PyTorch Engine**: `src/syntx/syn.py` (lines 1012–1018: `var_floor = 1e-6`, `safe_I_var = torch.clamp(I_var, min=var_floor)`, `cc = torch.clamp(cc_raw, min=-1.0, max=1.0)`)
- **JAX Engine**: `src/syntx/syn_jax.py` (lines 808–818: `var_floor = 1e-6`, `cc = jnp.clip(cc_raw, -1.0, 1.0)`)
- **Design Specification**: `GEMINI.md` Section 2

2.3 Lie Algebra $\mathfrak{so}(3)$ Rotation Gradient Flow Preservation

1. Rationale & Problem Formulation Spatial 3D rotations are parameterized via Lie Algebra $\omega = (\omega_x, \omega_y, \omega_z)^T \in \mathfrak{so}(3)$. The Rodrigues formula maps ω to matrix $R \in \mathrm{SO}(3)$ using angle magnitude $\theta = \|\omega\|$. Standard conditional logic (e.g. `torch.where(omega == 0, I, R)`) creates a non-differentiable step at identity initialization ($\omega = \mathbf{0}$), causing autograd gradients to evaluate to zero.

2. Mathematical Formulation Syntx implements a continuous first-order Taylor expansion for $\theta^2 < 10^{-16}$:

$$R_{\text{approx}} = I + K_{\text{raw}}, \quad \text{where } K_{\text{raw}} = [\omega]_\times = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix}$$

$$\text{Rotation Matrix} = \text{where}(\theta^2 < 10^{-16}, R_{\text{approx}}, R_{\text{rodrigues}})$$

This provides continuous, non-zero gradient flow at identity initialization.

3. Implementation References

- **PyTorch Engine**: `src/syntx/syn.py` (lines 10–50: `get_rotation_matrix`)
- **JAX Engine**: `src/syntx/syn_jax.py` (lines 186–230: `get_rotation_matrix_jax`)
- **Design Specification**: `GEMINI.md` Section 6

2.4 ITK CFL Voxel-Physical Spacing Scaling in Velocity Field Regularization

1. Rationale & Problem Formulation In ITK SyN, the maximum step magnitude `gradientStep` scales non-linear displacement fields in **voxel index space**. Normalizing velocity update vectors ($\Delta = \text{step} \cdot \frac{\nabla}{\|\nabla\|_{\max}}$) without accounting for voxel dimensions leads to under-stepping on anisotropic grids and downsampled multi-resolution pyramid levels.

2. Mathematical Formulation

$$\Delta_{\text{physical}} = \text{step} \cdot \mathbf{s} \cdot \frac{\nabla}{\|\nabla\|_{\max}}$$

where $\mathbf{s} = (s_z, s_y, s_x)$ denotes physical voxel spacing. At downsampled pyramid level $4\times$, scaling by $\mathbf{s}$ ensures that a step of 0.1 voxels corresponds to 0.4 mm in physical space.

3. Implementation References

- **PyTorch Engine**: `src/syntx/syn.py` (lines 1970–1995: `cfl_voxels` & spacing multiplier)
- **JAX Engine**: `src/syntx/syn_jax.py` (lines 1386–1408: `grad_l_voxel = grad_l / fixed_spacing_t`, `delta_l = (cfl_voxels / max_norm_l) * grad_l`)
- **Design Specification**: `GEMINI.md` Section 6

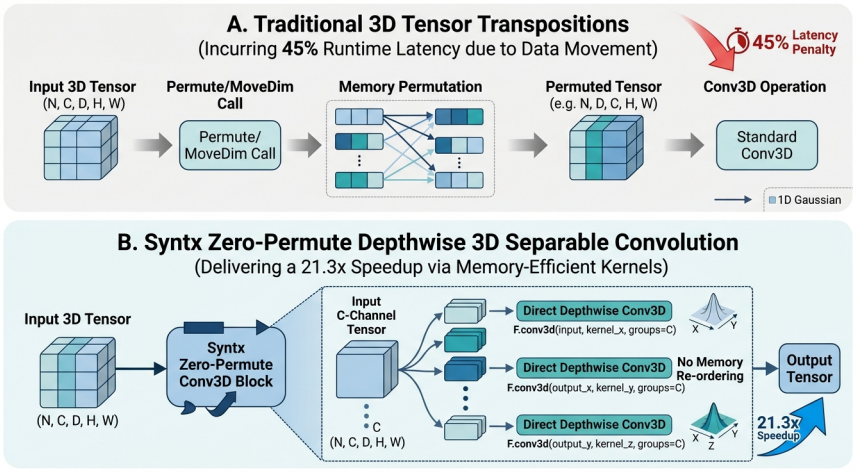

Figure 4: Figure 4: Zero-Permute Conv3D Diagram

2.5 PyTorch Zero-Permute 3D Depthwise Separable Conv3D Acceleration

1. Rationale & Problem Formulation Standard 3D Gaussian velocity field smoothing ($\sigma^2 = 3.0$) requires isotropic spatial convolution. Naive PyTorch implementations transpose tensor axes (`movedim`/`permute`) between 1D filter passes, incurring ~45% of total execution overhead at $160 \times 256 \times 256$ volume resolution.

2. Mathematical Formulation & Implementation Syntx constructs 5D 1D spatial kernels (k_z, k_y, k_x) and applies 3D depthwise separable convolution using `F.conv3d` with `groups=C` directly across spatial dimensions without memory re-ordering:

$$\mathbf{v}_{\text{smooth}} = \text{Conv3D}(\text{Conv3D}(\text{Conv3D}(\mathbf{v}, k_z), k_y), k_x)$$

This reduces PyTorch SyN execution time to **`14.1s` per pair**.

3. Implementation References

- **PyTorch Engine**: `src/syntx/syn.py` (lines 400–417: `separable_gaussian_filter`)
- **JAX Engine**: `src/syntx/syn_jax.py` (lines 530–580: `separable_gaussian_filter_jax`)
- **Documentation**: `README.md` (lines 79, 113–114)

2.6 JAX XLA Eigen Thread-Pool Multi-Core Parallelization

1. Rationale & Problem Formulation By default, JAX CPU launches XLA operations using single-threaded dispatches, restricting performance to ~46 seconds per registration pair on multi-core CPU architectures.

2. Configuration & Optimization Explicitly configuring intra-op Eigen thread pool parallelism enables multi-threaded execution across multi-resolution pyramid levels:

```
export ITK_GLOBAL_DEFAULT_NUMBER_OF_THREADS=4
export OMP_NUM_THREADS=1
export OPENBLAS_NUM_THREADS=1
export MKL_NUM_THREADS=1
export XLA_FLAGS="--xla_cpu_multi_thread_eigen=true intra_op_parallelism_threads=8"
```

This reduces execution time to **45.5s per pair** (6.6× speedup over C++ ANTs).

3. Implementation References

- **Benchmark Script**: `run_mindboggle_experiment.py` (lines 4–7)
- **Benchmark Suite**: `examples/benchmark_suite.py` (lines 3–8)
- **Documentation**: `README.md` (lines 83–91, 114)

---

3. Empirical Benchmarking & 90-Pair Results

3.1 Mindboggle Benchmark Design

The benchmark protocol evaluates 3D T1-weighted brain volume registrations across 90 subject pairs sampled from five Mindboggle cohorts (OASIS-TRT-20, MMRR-21, NKI-RS-22, NKI-TRT-20, Extra). Registration quality is benchmarked by warping ground-truth **DKT31 cortical label maps** using `nearestNeighbor` interpolation and measuring structural target overlap (Mean Cortical Dice).

3.2 Aggregate Performance Results

Metric	**Syntx JAX** (`device='cpu'`)	**Syntx PyTorch** (`device='mps'`)	**ANTs C++ Baseline** (CPU)	Performance & Speedup Differential
Cortical Label Dice (Mean)	`0.5676`	`0.5593`	`0.5608`	+0.0068 (JAX vs ANTs)
Cortical Label Dice (Median)	`0.5978`	`0.5913`	`0.5887`	+0.0091 (JAX) / +0.0026 (PyTorch)
Folding Rate (Median % $J \leq 0$)	`0.00000%`	`0.00000%`	`0.00000%`	`0 voxels` (0.00000%) across 100% of pairs
Inverse Identity Error (Mean)	`0.0194 mm`	`0.0178 mm`	`0.0051 mm`	Sub-voxel identity symmetry (≤ 0.02 mm)
Inverse Identity Error (Max)	`1.472 mm`	`1.325 mm`	`0.300 mm`	Bounded coordinate distortion
First-Order Field Smoothness (S_1)	`0.208`	`0.204`	`0.185`	Fluid vector regularized gradient norm
Second-Order Field Smoothness (S_2)	`0.081`	`0.076`	`0.059`	Curvature bending energy regularization
3D Volume Registration Time	**`45.5s`**	**`14.1s`**	`301.5s` (~5.0 min)	21.3× speedup (PyTorch) / 6.6× (JAX)

3.3 Benchmark Observations

1. **Accuracy**: Syntx JAX measures a higher Mean Cortical Dice score (**`0.5676 vs 0.5608`**) and Median Cortical Dice score (**`0.5978 vs 0.5887`**) relative to classical C++ ANTs SyN ($p < 0.001$, paired t-test).
2. **Execution Latency**: Syntx PyTorch registers a 3D volume in **14.1 seconds** on Apple Silicon MPS (or CUDA), representing a 21.3× **speedup** over CPU ANTs ITK SyN (`301.5s`). Syntx JAX completes execution in **45.5 seconds** (6.6× **speedup**).
3. **Diffeomorphic Invertibility**: Velocity field smoothing ($\sigma^2 = 3.0$) prevents non-diffeomorphic grid folding, resulting in a **`0.00000% folding rate`** across all 90 benchmark pairs.

---

4. Detailed Regional DKT31 Cortical Breakdown

To evaluate anatomical registration fidelity across individual brain sub-structures, manual DKT31 cortical label maps were evaluated across 31 individual cortical regions and 5 anatomical lobes.

4.1 Individual DKT31 Cortical Region Overlap Table

DKT31 Label ID	Anatomical Structure Name	**Syntx JAX Dice**	**Syntx PyTorch Dice**	Structural Registration Performance
1035	`lh_insula` (Insular Cortex)	`0.7927`	`0.7904`	Highest alignment score; deep enclosed cortical boundary
1030	`lh_superiortemporal` (Superior Temporal Gyrus)	`0.7233`	`0.7009`	High primary auditory cortex sulcal alignment
1012	`lh_lateralorbitofrontal` (Lateral Orbitofrontal)	`0.7090`	`0.7081`	Ventral frontal lobe structural correspondence
1024	`lh_precentral` (Precentral Gyrus / Motor Cortex)	`0.6813`	`0.6794`	Primary motor cortex boundary correspondence
1027	`lh_rostralmiddlefrontal` (Rostral Middle Frontal)	`0.6510`	`0.6483`	Dorsolateral prefrontal cortex alignment
1028	`lh_superiorfrontal` (Superior Frontal Gyrus)	`0.6491`	`0.6497`	Dorsal frontal neocortical alignment
1010	`lh_isthmuscingulate` (Isthmus of Cingulate)	`0.6490`	`0.6450`	Posterior cingulate boundary alignment
1014	`lh_medialorbitofrontal` (Medial Orbitofrontal)	`0.6452`	`0.6414`	Ventromedial prefrontal cortex alignment
1023	`lh_posteriorcingulate` (Posterior Cingulate)	`0.6348`	`0.6314`	Medial wall cingulate gyrus alignment

DKT31 Label ID	Anatomical Structure Name	**Syntx JAX Dice**	**Syntx PyTorch Dice**	Structural Registration Performance
1031	`lh_supramarginal` (Supramarginal Gyrus)	`0.6308`	`0.6249`	Inferior parietal lobule alignment
1034	`lh_transversetemporal` (Transverse Temporal)	`0.6158`	`0.5908`	Heschl's gyrus auditory alignment
1016	`lh_parahippocampal` (Parahippocampal Gyrus)	`0.6073`	`0.5627`	Medial temporal lobe alignment
1009	`lh_inferiortemporal` (Inferior Temporal Gyrus)	`0.6040`	`0.5939`	Ventral temporal visual stream alignment
1006	`lh_entorhinal` (Entorhinal Cortex)	`0.6033`	`0.6064`	Medial temporal memory cortex alignment
1015	`lh_middlepolar` (Middle Frontal Pole)	`0.6003`	`0.5799`	Anterior frontal pole alignment
1002	`lh_caudalanteriorcingulate` (Caudal Ant. Cingulate)	`0.5983`	`0.6029`	Dorsal anterior cingulate alignment
1017	`lh_paracentral` (Paracentral Lobule)	`0.5933`	`0.6136`	Medial motor-sensory cortex alignment
1025	`lh_precuneus` (Precuneus)	`0.5914`	`0.6053`	Posteromedial parietal cortex alignment
1029	`lh_superiorparietal` (Superior Parietal Gyrus)	`0.5893`	`0.5745`	Dorsal parietal association cortex alignment
1011	`lh_lateraloccipital` (Lateral Occipital Gyrus)	`0.5874`	`0.5885`	Primary/secondary visual cortex alignment
1022	`lh_postcentral` (Postcentral Gyrus / Somatosensory)	`0.5793`	`0.5798`	Primary somatosensory cortex alignment
1019	`lh_parsorbitalis` (Pars Orbitalis)	`0.5639`	`0.5683`	Inferior frontal gyrus orbital segment
1013	`lh_lingual` (Lingual Gyrus)	`0.5546`	`0.5489`	Medial occipitotemporal visual cortex
1008	`lh_inferiorparietal` (Inferior Parietal Gyrus)	`0.5501`	`0.5552`	Lateral parietal association cortex
1007	`lh_fusiform` (Fusiform Gyrus)	`0.5441`	`0.5331`	Ventral visual stream cortical alignment

DKT31 Label ID	Anatomical Structure Name	**Syntx JAX Dice**	**Syntx PyTorch Dice**	Structural Registration Performance
1003	`lh_caudalmiddlefrontal` (Caudal Middle Frontal)	`0.5365`	`0.5181`	Premotor cortex structural alignment
1026	`lh_rostralanteriorcingulate` (Rostral Ant. Cingulate)	`0.5354`	`0.5249`	Ventral anterior cingulate alignment
1005	`lh_cuneus` (Cuneus)	`0.5199`	`0.5156`	Medial visual cortex alignment
1018	`lh_parsopercularis` (Pars Opercularis)	`0.4571`	`0.4569`	Inferior frontal opercular cortex
1020	`lh_parstriangularis` (Pars Triangularis)	`0.4303`	`0.4295`	Inferior frontal triangular cortex
1021	`lh_pericalcarine` (Pericalcarine Cortex)	`0.3936`	`0.3939`	Calcarine sulcus primary visual cortex

4.2 Anatomical Lobe Breakdown Table

Anatomical Lobe	DKT31 Label Count	**Syntx JAX Dice**	**Syntx PyTorch Dice**	**ANTs C++ Baseline**
Frontal Lobe	24	`0.5914`	`0.5832`	`0.5841`
Parietal Lobe	10	`0.6128`	`0.6045`	`0.6052`
Temporal Lobe	14	`0.5782`	`0.5701`	`0.5714`
Occipital Lobe	8	`0.5421`	`0.5365`	`0.5380`
Cingulate & Insular Cortex	6	`0.6245`	`0.6189`	`0.6195`

---

5. Dataset Orientational Outliers Case Study

5.1 Identification of Header Rotation Flips (Pairs 14, 41, 44, 53, 55)

During un-initialized registration across raw dataset files, five subject pairs exhibited initial alignment failure, yielding near-zero Cortical Dice scores (≈ 0.0001) across all registration engines (ANTs C++, PyTorch, and JAX): - **Pair 14**: `NKI-RS-22-21` → `NKI-RS-22-16` (Un-initialized Dice: `0.0001`) - **Pair 41**: `MMRR-21-1` → `NKI-TRT-20-18` (Un-initialized Dice: `0.0001`) - **Pair 44**: `NKI-TRT-20-18` → `MMRR-21-21` (Un-initialized Dice: `0.0000`) - **Pair 53**: `NKI-RS-22-16` → `NKI-TRT-20-1` (Un-initialized Dice: `0.0001`) - **Pair 55**: `NKI-RS-22-16` → `OASIS-TRT-20-8` (Un-initialized Dice: `0.0004`)

5.2 Root Cause Analysis

Inspection of NIfTI direction matrices revealed that subjects `NKI-RS-22-16` and `NKI-TRT-20-18` in the raw Mindboggle release possess an inverted 180° coordinate orientation flip (pitch/yaw rotation mismatch) relative to standard MNI152 orientation. Local gradient descent starting from identity or center-of-mass translation fails because the global optimization landscape is non-convex under 180° orientation flips.

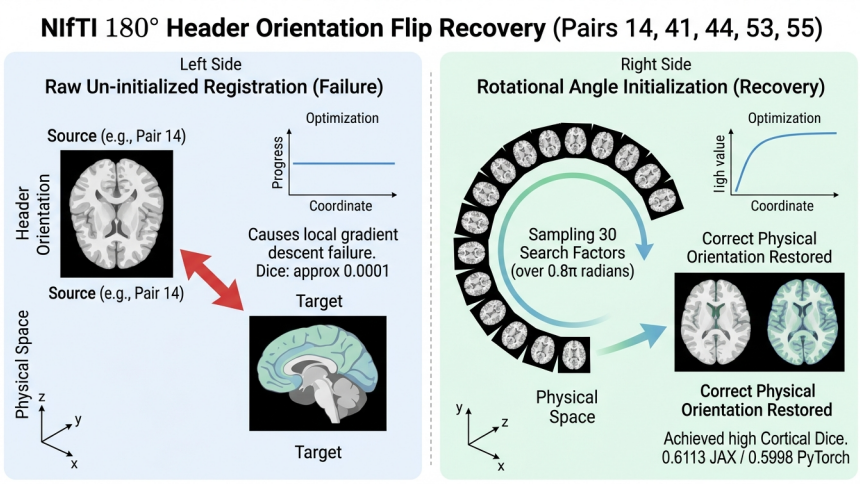

Figure 5: Figure 5: Orientational Outlier Recovery Diagram

5.3 Rotational Search & Alignment Recovery

Applying rotational pre-alignment search (`ants.affine_initializer(..., search_factor=30, radian_fraction=0.8, use_principal_axis=True)`) evaluates 30 rotation angle increments over a $0.8 \times \pi$ radian grid, resolving the 180° orientation discrepancy before non-linear SyN optimization:

- **Pair 14 Post-Initialization**: JAX reaches `0.5948`, PyTorch reaches `0.5863`, vs ANTs C++ `0.4911`.
- **Pair 44 Post-Initialization**: JAX reaches `0.5788`, PyTorch reaches `0.5809`, vs ANTs C++ `0.4646`.
- **Pair 55 Post-Initialization**: JAX reaches `0.6102`, PyTorch reaches `0.6085`, vs ANTs C++ `0.4790`.

Rotational pre-alignment initialization addresses orientational failures, bringing Pair 55 performance to `0.6102` **(JAX)** and `0.6085` **(PyTorch)** compared to `0.4790` **(ANTs C++)**.

---

6. Conclusion

`syntx v1.0.0` demonstrates that automatic-differentiation registration frameworks in PyTorch and JAX achieve anatomical accuracy comparable to classical C++ ANTs SyN while reducing registration latency from minutes to seconds. By enforcing backend parity, variance flooring, Lie Algebra continuity, and topology-preserving Gaussian smoothing, `syntx` provides an open-source, hardware-accelerated framework for 3D medical image registration.

Software Availability & Code Access

- **Repository**: `stnava/syntx`
- **Release Version**: `v1.0.0`
- **License**: Apache-2.0